Git for devops

1. file system vs git
2. git init, git status
3. untracked, staged, tracked -->files
4. git add <filename>
5. git rm --cached <filename>
5a. git rm <filename> -- to delete the file from local not from central(to delete from central git commit and git push you have to do)
6. git commit -m "added test"

Show more

Absolutely. I've reorganized your points into **proper Git + DevOps notes**, corrected a few inaccurate statements, and added the practical purpose of each command.

> **Important security note:** You pasted a GitHub Personal Access Token in your message. Treat it as **compromised**. Revoke/delete that token in GitHub and generate a new one if you still need it. **Never put a token directly in a Git remote URL or share it in notes.**

Edit

Git for DevOps — Complete Notes

1. What is Git?

Git is a distributed version control system used to:

- Track changes in source code
- Maintain different versions of files
- Work collaboratively with multiple developers
- Create branches for different features/environments
- Merge changes from different developers
- Revert unwanted changes
- Connect local development with remote repositories such as GitHub

Git vs File System

File System	Git

Stores files	Stores files + their history
No built-in version tracking	Tracks every committed version
Difficult to identify who changed what	Shows author and changes
No easy rollback	Can restore/revert previous versions
Collaboration is difficult	Designed for collaboration
No branching concept	Supports branches

Key idea:

> File system = stores the current files
> Git = stores the files + their history

---

2. Git Repository

A **Git repository** is a directory whose changes are managed by Git.

There are two major repositories:

Local Repository

Repository on your computer.

Remote Repository

Repository hosted on a server such as GitHub.

```
Developer's Computer
        |
        | Git commands
        ↓
 Local Git Repository
        |
        | push / pull / fetch
        ↓
 GitHub Remote Repository
```

---

3. git init

```
git init
```

Purpose

Creates a new Git repository in the current directory.

It creates a hidden `.git` directory.

```
project/
├── file1.txt
├── file2.txt
└── .git/
```

The `.git` directory contains Git's internal information, including:

- commits
- branches
- configuration
- references
- repository history

Check it

```
ls -a
```

You should see:

```
.git
```

---

4. git status

```
git status
```

Shows the current state of your working directory.

It tells you:

- Which files are modified
- Which files are untracked
- Which files are staged
- Current branch

- Changes ready for commit

Very important command

When working with Git, **`git status` should be one of your most frequently used commands.**

---

5. Git File States

A file generally moves through these states:

```
Untracked
    ↓
git add
    ↓
Staged
    ↓
git commit
    ↓
Tracked / Committed
```

More accurately, Git works with three important areas:

```
Working Directory
       ↓ git add
Staging Area
       ↓ git commit
Local Repository
       ↓ git push
Remote Repository
```

Untracked

Git sees the file but is **not tracking it yet**.

Example:

```
?? test.txt
```

Staged

The changes have been selected for the next commit.

```
git add test.txt
```

Tracked

Git is already managing the file.

A tracked file can be:

- Modified
- Staged
- Committed

---

6. git add

```
git add filename
```

Moves the file's current changes from the **working directory → staging area**.

Example:

```
git add test.txt
```

To stage everything:

```
git add .
```

Why staging exists?

It allows you to choose exactly which changes should go into the next commit.

Example:

```
file1.txt → changed
file2.txt → changed

git add file1.txt

Next commit:
file1.txt → included
file2.txt → not included
```

---

7. git rm --cached

```
git rm --cached filename
```

Removes a file from Git's **staging/tracking**, while keeping the physical file in your local directory.

Example:

```
git add password.txt
git rm --cached password.txt
```

Now:

```
password.txt → remains on computer
password.txt → removed from Git tracking
```

This is commonly useful when you accidentally added a file that should be ignored.

Usually you would then add it to `.gitignore`.

---

8. git rm

```
git rm filename
```

Removes the file from:

1. Working directory
2. Git staging area

Example:

```
git rm test.txt
```

Important correction

`git rm` does **not immediately delete the file from GitHub**.

To make the deletion appear in the remote repository:

```
git rm test.txt
git commit -m "Remove test file"
git push origin main
```

So:

```
git rm
   ↓
commit
   ↓
push
   ↓
Remote repository updated
```

---

9. git commit

```
git commit -m "added test"
```

Creates a permanent snapshot of the staged changes in the **local Git repository**.

Example:

```
git add test.txt
git commit -m "Add test file"
```

Important

A commit does **not** automatically send code to GitHub.

```
git add
   ↓
git commit
   ↓
Local repository

git push
   ↓
GitHub
```

Good commit messages

```
git commit -m "Add login functionality"
git commit -m "Fix database connection"
git commit -m "Update Docker configuration"
```

Avoid:

```
git commit -m "changes"
git commit -m "abc"
```

10. git restore

```
git restore filename
```

Discards **unstaged changes** in a tracked file and restores it to the version in the staging area.

Example:

```
git restore test.txt
```

Suppose:

```
Original:
Hello

Modified:
Hello World
```

After:

```
git restore test.txt
```

It becomes:

```
Hello
```

Important

`git restore filename` is mainly used to discard **working-directory changes**.

It does not work as a way to recover an untracked file.

---

11. git restore --staged

```
git restore --staged filename
```

Moves a file:

```
Staged → Unstaged
```

It **does not delete your changes**.

Example:

```
git add test.txt
git restore --staged test.txt
```

Now the file remains modified, but it is no longer staged.

Remember

```
git restore file
        ↓
Discard working-directory changes


git restore --staged file
        ↓
Unstage the file
```

---

12. GitHub Repository

GitHub is a remote platform that hosts Git repositories.

Typical workflow:

```
Create project locally
        ↓
git init
        ↓
git add
        ↓
git commit
        ↓
Create GitHub repository
        ↓
Connect local repository
        ↓
git push
```

---

13. git remote

To connect your local Git repository to GitHub:

```
git remote add origin <repository-url>
```

Example:

```
git remote add origin https://github.com/username/project.gi
```

Meaning

```
remote → remote repository configuration
add    → add a remote
origin → name given to the remote
URL    → GitHub repository address
```

`origin` is simply the conventional name for the main remote repository.

Check remote

```
git remote -v
```

---

14. git branch -M main

```
git branch -M main
```

Renames the current branch to:

```
main
```

Important correction

Do **not** assume the default local branch is always `master`.

Modern Git installations commonly use `main`, and the initial branch name can also be configured.

You can check your current branch using:

```
git branch
```

---

15. GitHub Personal Access Token (PAT)

GitHub may require authentication when pushing over HTTPS.

A **Personal Access Token (PAT)** can be used instead of a password.

For security:

- Never share your PAT.
- Never commit it into source code.
- Never put it into scripts or `.gitignore`-omitted files.
- Prefer GitHub CLI, credential managers, or SSH keys for regular development.

If a token is accidentally exposed

Immediately:

```
Revoke the token
        ↓
Generate a new token if required
        ↓
Update your authentication method
```

Never store a real token in study notes.

---

16. git push

```
git push origin main
```

Uploads local commits to the remote repository.

```
Local Repository
      |
      | git push
      ↓
GitHub Repository
```

Typical workflow

```
git add .
git commit -m "Add feature"
git push origin main
```

Important

Before pushing, your local repository should contain the changes/commits you intend to publish.

For team environments, first synchronize with the remote when appropriate:

```
git pull
```

Then resolve any conflicts and push your work.

---

17. Fork vs Clone

These are different concepts.

Fork

A **fork** creates your own copy of another person's GitHub repository under your GitHub account.

Commonly used in open-source projects.

```
Original Repository
        ↓
      Fork
        ↓
Your GitHub Repository
```

Clone

```
git clone <repository-url>
```

Downloads a remote Git repository to your computer.

```
GitHub Repository
       ↓
   git clone
       ↓
Local Computer
```

Typical open-source workflow

```
Fork
 ↓
Clone
 ↓
Create branch
 ↓
Make changes
 ↓
Commit
```

```
 ↓
Push
 ↓
Pull Request
```

---

18. .gitignore

`.gitignore` tells Git which files/folders should **not be tracked**.

Example:

```
node_modules/
.env
*.log
target/
.idea/
.vscode/
```

Common things to ignore:

- Passwords/secrets
- Environment files
- Build output
- IDE configuration
- Dependency directories
- Log files

Important

`.gitignore` does not magically remove a file that Git is already tracking.

If a file is already tracked:

```
git rm --cached filename
```

Then add it to `.gitignore`.

---

19. git pull

```
git pull
```

Gets changes from the remote repository and integrates them into your current local branch.

Conceptually:

```
git pull
≈
git fetch + git merge
```

Depending on configuration, pull can also integrate using rebase.

Example:

```
git pull origin main
```

Why use it?

Suppose another developer pushed:

```
Developer A → GitHub
```

Your local repository is now behind.

```
git pull
```

brings the remote changes into your local branch.

Possible problem

A pull can produce a **merge conflict** if your local changes and remote changes cannot be automatically combined.

---

20. git branch

```
git branch
```

Shows local branches.

Example:

```
* main
  dev
  feature-login
```

`*` indicates the currently checked-out branch.

---

21. Creating a Branch

```
git branch dev
```

Creates a branch called:

```
dev
```

It does **not** switch to that branch.

Create and switch in one command:

```
git switch -c dev
```

---

22. git switch

```
git switch dev
```

Switches to an existing branch.

Example:

```
git switch dev
```

You are now working on the `dev` branch.

Create + switch

```
git switch -c feature-login
```

---

23. git checkout

Older/common command:

```
git checkout dev
```

Can switch branches.

It can also create and switch to a branch:

```
git checkout -b dev
```

Modern recommendation

For branch operations, prefer:

```
git switch
```

because Git provides separate commands for branch switching and file restoration.

---

24. Branching Strategy

A team may use branches such as:

```
main
 │
 ├── development
 │
 ├── staging
 │
 └── feature/login
```

A simplified environment-based model could be:

```
feature → development → staging → main
```

However, **branch names and deployment strategies vary between organizations**.

For example:

```
main       → production
develop    → development
staging    → testing/pre-production
feature/*  → individual features
```

The important concept is:

> Branches allow developers to work independently without directly changing the main codebase.

---

25. HEAD

HEAD is a reference to the commit/branch you are currently working from.

Usually:

```
HEAD
 ↓
current branch
 ↓
latest commit on that branch
```

Example:

```
A → B → C
        ↑
      HEAD
```

If you create another commit:

```
A → B → C → D
            ↑
          HEAD
```

Remember

> HEAD tells Git where your current position is.

---

26. git log

```
git log
```

Shows commit history.

Example information:

```
commit ID
Author
Date
Commit message
```

Useful for understanding what happened in the repository.

---

27. git log --oneline

```
git log --oneline
```

Displays a compact commit history.

Example:

```
a83f2c1 Add login functionality
b91d821 Fix database connection
c72a190 Initial commit
```

This is easier to scan than the normal `git log`.

---

28. Merge

Suppose:

```
main
  A---B


dev
      \
       C---D
```

You want the changes from `dev` in `main`.

You can merge:

```
git switch main
git merge dev
```

Conceptually:

```
dev
  C---D
      \
main   M
```

The exact history depends on whether Git performs a fast-forward or creates a merge commit.

---

29. Pull Request (PR)

A **Pull Request** is a request to merge changes from one branch/repository into another.

Example:

```
dev
 ↓
Pull Request
 ↓
main
```

On GitHub:

```
New Pull Request
Base: main
Compare: dev
```

Then reviewers can:

- Review code
- Comment
- Request changes
- Approve
- Merge

Important

A **Pull Request is a GitHub collaboration feature**, not a Git command.

---

30. git fetch

```
git fetch
```

Downloads information about changes from the remote repository **without automatically merging those changes into your current branch**.

Example:

```
git fetch origin
```

After fetching:

```
Remote information
        ↓
Local remote-tracking branches
```

You can inspect the changes before deciding what to do.

Fetch vs Pull

```
git fetch
→ Download remote changes
→ Don't automatically merge

git pull
→ Fetch + integrate changes
```

---

31. Git Merge Conflict

A conflict occurs when Git cannot automatically combine changes.

Example:

Developer A:

```
print("Hello")
```

Developer B changes the same line:

```
print("Hi")
```

Git may produce:

```
<<<<<<< HEAD
print("Hello")
=======
print("Hi")
>>>>>>> branch-name
```

Meaning

```
<<<<<<<
Your/current branch

=======
Separator

>>>>>>>
Incoming branch
```

Resolve

Manually decide what the final code should be.

For example:

```
print("Hello everyone")
```

Then:

```
git add filename
git commit -m "Resolve merge conflict"
```

If the conflict occurred during a merge, you may then push the resulting commit.

Important

Do **not** blindly remove conflict markers without deciding which code should remain.

---

32. Hidden Files

On Linux/macOS, files beginning with `.` are hidden.

Example:

```
.git
.gitignore
```

Use:

```
ls -a
```

to display hidden files.

For Windows File Explorer:

```
View
  ↓
Show
  ↓
Hidden items
```

---

33. Git GUI

Git can be operated through graphical tools instead of only the command line.

Examples include:

- GitHub Desktop
- IDE Git integrations
- Other Git GUI clients

GUI tools allow you to:

- View changes
- Stage files
- Commit
- Create branches
- Merge
- Push
- Pull
- View history

DevOps takeaway

You should still understand the **Git commands underneath the GUI**, because DevOps automation normally uses command-line Git.

---

34. Git Hooks

Git Hooks are scripts that Git automatically executes at specific points in the Git workflow.

Examples:

```
pre-commit
commit-msg
pre-push
post-commit
```

Example

A `pre-commit` hook can:

```
Developer runs git commit
          ↓
```

```
     pre-commit hook
          ↓
Run tests / lint / validation
          ↓
If successful → commit
If failed → stop commit
```

This helps prevent bad code from being committed.

---

35. Creating a pre-commit Hook

Inside a Git repository:

```
cd .git/hooks
```

You may see files such as:

```
pre-commit.sample
pre-push.sample
commit-msg.sample
```

A hook named:

```
pre-commit
```

can contain shell commands.

Example:

```
#!/bin/sh

echo "Running checks..."

# Run your validation commands here
```

Then make it executable:

```
chmod +x pre-commit
```

Important correction

Avoid:

```
chmod 777 pre-commit
```

`777` gives read/write/execute permission to everyone and is usually unnecessarily permissive.

Prefer:

```
chmod +x pre-commit
```

---

36. Shell Script

A **shell script** is a file containing commands that can be executed by a shell.

Example:

```
#!/bin/sh

echo "Running tests..."
echo "Build started..."
```

Git hooks commonly use shell scripts.

DevOps connection

Shell scripting + Git hooks can automate:

```
Validation
Testing
Linting
Formatting
Build checks
```

---

37. GitHub Actions

GitHub Actions is GitHub's automation/CI/CD platform.

It can automatically run workflows when events occur.

For example:

```
Developer pushes code
        ↓
GitHub Actions starts
        ↓
```

```
Build
        ↓
Test
        ↓
Lint
        ↓
Package
        ↓
Deploy
```

Common uses:

- Continuous Integration (CI)
- Continuous Delivery (CD)
- Automated testing
- Building applications
- Docker image creation
- Deployment
- Code quality checks

Workflow files are generally stored in:

```
.github/workflows/
```

Example:

```
.github/
└── workflows/
    └── ci.yml
```

---

38. Pylint

Pylint is a static code analysis tool for Python.

It checks Python code for issues such as:

- Errors
- Coding-standard violations
- Unused variables
- Poor naming
- Potential bugs
- Code-quality problems

Example:

```
pylint app.py
```

Git/DevOps connection

Pylint can be integrated into:

```
Git Hook
    ↓
Local code-quality check
```

or:

```
GitHub Push
    ↓
GitHub Actions
    ↓
Pylint
    ↓
Pass / Fail
```

This is an example of **automated code quality checking**.

---

39. Complete Git Workflow

A basic individual developer workflow:

```
Create project
     ↓
git init
     ↓
Create/modify files
     ↓
git status
     ↓
git add
     ↓
git commit
     ↓
Connect GitHub
     ↓
git push
```

For an existing project:

```
git clone
    ↓
git switch -c feature
    ↓
Modify code
    ↓
git status
    ↓
git add
    ↓
git commit
    ↓
git push
    ↓
Pull Request
    ↓
Code Review
    ↓
Merge
```

---

40. Most Important Git Commands

Command	Purpose
`git init`	Create a Git repository
`git status`	Check repository status
`git add file`	Stage a file
`git add .`	Stage changes
`git restore file`	Discard unstaged changes
`git restore --staged file`	Unstage a file
`git rm file`	Delete file and stage deletion
`git rm --cached file`	Stop tracking while keeping local file

`git commit -m "message"`	Create a commit
`git log`	View commit history
`git log --oneline`	Compact commit history
`git branch`	List branches
`git branch dev`	Create branch
`git switch dev`	Switch branch
`git switch -c dev`	Create + switch branch
`git checkout dev`	Switch branch (older/common syntax)
`git merge dev`	Merge dev into current branch
`git fetch`	Download remote changes without integrating
`git pull`	Fetch + integrate remote changes
`git push`	Upload commits to remote
`git clone URL`	Copy remote repository locally
`git remote -v`	Show remote repositories

---

41. The Four Places You Must Understand

This is one of the **most important concepts for interviews and practical Git usage**.



```
                    | git push
                    ↓
            Remote Repository
                    |
                    | git fetch/pull
                    ↓
                  Local
```

Remember:

Working Directory

→ Where you edit files.

Staging Area

→ Where you select changes for the next commit.

Local Repository

→ Where your commits are stored locally.

Remote Repository

→ Shared repository hosted on GitHub/GitLab/Bitbucket.

---

42. Most Important Differences

`git add` **vs** `git commit`

```
git add
→ Working Directory → Staging Area

git commit
→ Staging Area → Local Repository
```

`git push` **vs** `git pull`

```
git push
→ Local → Remote

git pull
→ Remote → Local + integration
```

`git fetch` **vs** `git pull`

```
git fetch
→ Download remote information
→ Don't automatically integrate

git pull
→ Fetch + integrate
```

`git rm` vs `git rm --cached`

```
git rm file
→ Remove from Git + local working directory

git rm --cached file
→ Remove from Git tracking
→ Keep local file
```

`git restore` vs `git restore --staged`

```
git restore file
→ Discard unstaged changes

git restore --staged file
→ Unstage changes
→ Keep the file changes
```

---

43. DevOps Perspective

Git is not just a version-control tool. It is a **foundation of modern DevOps workflows**.

A typical DevOps pipeline can look like:

```
Developer
    ↓
Git Branch
    ↓
Commit
    ↓
Push
    ↓
GitHub
    ↓
Pull Request
```

```
    ↓
Code Review
    ↓
GitHub Actions
    ↓
Build
    ↓
Pylint / Quality Checks
    ↓
Automated Tests
    ↓
Docker Build
    ↓
Deployment
```

Therefore, you should understand these Git concepts especially well:

1. **Repository**
2. **Working Directory**
3. **Staging Area**
4. **Commit**
5. **Branch**
6. **Merge**
7. **Pull Request**
8. **Remote**
9. **Fetch**
10. **Pull**
11. **Push**
12. **Conflict**
13. **Git Hooks**
14. **GitHub Actions**

---

44. Quick Memory Sheet

```
git init
→ Start Git

git status
→ What's happening?
```

```
git add
→ Select changes

git commit
→ Save snapshot locally

git push
→ Send commits to GitHub

git fetch
→ Bring remote information

git pull
→ Bring + integrate remote changes

git clone
→ Download repository

git branch
→ Manage branches

git switch
→ Change branch

git merge
→ Combine branches

git restore
→ Discard working changes

git restore --staged
→ Unstage changes

git rm
→ Delete + stage deletion

git rm --cached
→ Stop tracking, keep local file

git log
→ See history

.gitignore
```

```
→ Tell Git what not to track

Git Hook
→ Run automation at Git events

GitHub Actions
→ Automate CI/CD
```

Final Takeaway

The most important Git flow to remember is:

```
EDIT
 ↓
git status
 ↓
git add
 ↓
git commit
 ↓
git push
```

And for team development:

```
CLONE
 ↓
BRANCH
 ↓
EDIT
 ↓
ADD
 ↓
COMMIT
 ↓
PUSH
 ↓
PULL REQUEST
 ↓
REVIEW
 ↓
MERGE
 ↓
DEPLOY
```

One-line concept to remember:

> **Git tracks changes, commits create snapshots, branches isolate work, merges combine work, and remote repositories enable collaboration.**

One correction especially worth remembering for your DevOps notes: `git pull origin master` **is not universally correct**—use the branch your repository actually uses, commonly `main` today. Also, a GitHub **Pull Request is not the same thing as** `git pull`; they are completely different concepts.

ChatGPT can make mistakes. Check important info.